



SPRING SECURITY CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SECURITY FILTER CHAIN

Installation

```
@Bean SecurityFilterChain chain(HttpSecurity ...
return http
    .authorizeHttpRequests(a ->
        a.requestMatchers("/public/**").permitAll()
        .anyRequest().authenticated())
    .build();
}
```

04. JWT FILTER

Workflow

```
class JwtFilter extends OncePerRequestFilter{
    void doFilterInternal(req, res, chain){
        String token = extractToken(req);
        if(jwtUtil.validate(token)){
            var auth = jwtUtil.getAuthentication(token);
            SecurityContextHolder.getContext().setAuthent...
        }
        chain.doFilter(req, res);
    }
}
```

02. USERDETAILSSERVICE

Architecture

```
@Service class UserSvc implements UserDetails...
public UserDetails loadUserByUsername(String ...
    User user = repo.findByEmail(u);
    return new org.springframework.security.core
        .userdetails.User(user.email, user.pwHash,
        mapRoles(user.roles));
}
```

05. OAUTH2 / OIDC

CRUD API

```
<dep>spring-boot-starter-oauth2-client</dep>
spring.security.oauth2.client.registration
    .google.client-id=...
    .google.client-secret=...
    http.oauth2Login(withDefaults())
```

OAuth2User / OidcUser principals available

03. PASSWORD ENCODING

YAML / Config

```
@Bean PasswordEncoder encoder(){
    return new BCryptPasswordEncoder();
}
encoder.encode(rawPassword)
encoder.matches(raw, stored)
```

BCrypt strength 1012; use Argon2 for new projects

06. METHOD SECURITY

Integration

```
@EnableMethodSecurity
@PreAuthorize("hasRole('ADMIN')")
@PostAuthorize("returnObject.owner == authent...
@Secured({"ROLE_USER", "ROLE_ADMIN"})
@RolesAllowed("ADMIN") (JSR-250)
```



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. CORS & CSRF

Hardening

```
http.cors(cors -> cors.configurationSource(so...
CorsConfiguration cfg = new CorsConfiguration...
cfg.addAllowedOrigin("https://myapp.com");
```

CSRF enabled by default; disable for stateless APIs:

```
http.csrf(AbstractHttpConfigurer::disable)
```

10. SECURITY CONTEXT

Unit Tests

```
Authentication auth = SecurityContextHolder
.getContext().getAuthentication();
String name = auth.getName();
Collection<? extends GrantedAuthority> roles ...
@AuthenticationPrincipal UserDetails user in...
```

08. RBAC PATTERN

Observability

```
.requestMatchers("/admin/**").hasRole("ADMIN")
.requestMatchers("/api/**").hasAuthority("SCO...
```

Role = ROLE_ prefix; Authority = any string

GrantedAuthority interface

Hierarchy: RoleHierarchyImpl for nested roles

11. TESTING

Commands

```
@WithMockUser(roles="ADMIN")
@WithUserDetails("admin@example.com")
mvc.perform(get("/admin")).andExpect(status()...)
@WithSecurityContext for custom auth
SecurityMockMvcRequestPostProcessors.csrf() f...
```

09. EXCEPTION HANDLING

Optimization

```
http.exceptionHandling(e ->
e.authenticationEntryPoint(myEntryPoint)
.accessDeniedHandler(myHandler))
```

401 Unauthorized vs 403 Forbidden

```
AuthenticationEntryPoint: commence(req,res,ex)
AccessDeniedHandler: handle(req,res,ex)
```

12. COMMON GOTCHAS

Warning

CSRF must be disabled for REST APIs with JWT
Roles need ROLE_ prefix in hasRole()
SecurityContextHolder clears after request by default
Don't store sensitive data in JWT it's not encrypted
HTTPS required in prod; use HSTS header